

Westin Gjervold

ComSci 342 Homework 4

1. (Function Recursion) reverse.scm

```
(define reverse
  (lambda (lst)
    (if (null? lst)
        (list)
        (append (reverse (cdr lst)) (list (car lst))))))
```

Uses append function:

```
(define append
  (lambda (a b)
    (if (null? a) b (cons (car a) (append (cdr a) b)))))
```

2. (List)

a. compList.scm

```
(define compList
  (lambda (lst)
    (if (null? lst)
        (list)
        (let ((rest (compList (cdr lst))))
          (if (null? rest)
              (list (car lst))
              (if (= (car lst) (car rest))
                  rest
                  (cons (car lst) rest)))))))
```

b. pell.scm (with helper functions (uses reverse and append from Q1))

```
(define reverse
  (lambda (lst)
    (if (null? lst)
        (list)
        (append (reverse (cdr lst)) (list (car lst))))))

(define append
  (lambda (a b)
    (if (null? a) b (cons (car a) (append (cdr a) b)))))

(define nextPell
  (lambda (p0 p1)
    (+ (* 2.0 p1) p0)))
```

```

(define pellLoop
  (lambda (rem p0 p1 accRev)
    (if (= rem 0)
        (reverse accRev)
        (pellLoop (- rem 1)
                    p1
                    (nextPell p0 p1)
                    (cons (nextPell p0 p1) accRev)))))

(define pell
  (lambda (n)
    (if (= n 0)
        (list)
        (if (= n 1)
            (list 0.0)
            (pellLoop (- n 2) 0.0 1.0 (cons 1.0 (cons 0.0 (list))))))))

```

c. isSorted.scm (with helper function)

```

(define isSortedHelper
  (lambda (lst dir)
    (if (null? lst)
        #t
        (if (null? (cdr lst))
            #t
            (if (= (car lst) (car (cdr lst)))
                (isSortedHelper (cdr lst) dir)
                (if (= dir 0)
                    (if (< (car lst) (car (cdr lst)))
                        (isSortedHelper (cdr lst) 1)
                        (isSortedHelper (cdr lst) -1))
                    (if (= dir 1)
                        (if (< (car lst) (car (cdr lst)))
                            (isSortedHelper (cdr lst) 1)
                            #f)
                        (if (> (car lst) (car (cdr lst)))
                            (isSortedHelper (cdr lst) -1)
                            #f))))))))))

(define isSorted
  (lambda (lst)
    (isSortedHelper lst 0)))

```

3. (Lists and Pairs) combine.scm

a. Defining student_grades list

```

(define student_grades
  (list (cons 0 90) (cons 1 85) (cons 2 92)))

```

b. get-scores function

```
(define get_scores
  (lambda (pairs)
    (if (null? pairs)
        (list)
        (cons (cdr (car pairs))
              (get_scores (cdr pairs))))))
```

c. combine function (with some helper functions)

```
(define sameLength
  (lambda (a b)
    (if (null? a)
        (if (null? b)
            #t
            #f)
        (if (null? b)
            #f
            (sameLength (cdr a) (cdr b))))))

(define combineHelper
  (lambda (ids ages)
    (if (null? ids)
        (list)
        (cons (cons (car ids) (car ages))
              (combineHelper (cdr ids) (cdr ages))))))

(define combine
  (lambda (ids ages)
    (if (sameLength ids ages)
        (combineHelper ids ages)
        (list))))
```

4. (Higher-order Functions) pred.scm

```
(define filter_list
  (lambda (pred lst)
    (if (null? lst)
        (list)
        (let ((x (car lst)))
          (rest (filter_list pred (cdr lst)))))
    (if (pred x)
        (cons x rest)
        rest))))
```

5. (Higher-order Functions) procList.scm

a. procList function (With helper functions (sameLength function from Q3c))

```
(define sameLength
  (lambda (a b)
    (if (null? a)
        (if (null? b)
            #t
            #f)
        (if (null? b)
            #f
            (sameLength (cdr a) (cdr b))))))

(define procListHelper
  (lambda (op lst1 lst2)
    (if (null? lst1)
        (list)
        (cons (op (car lst1) (car lst2))
              (procListHelper op (cdr lst1) (cdr lst2))))))

(define procList
  (lambda (op lst1 lst2)
    (if (sameLength lst1 lst2)
        (procListHelper op lst1 lst2)
        (list))))
```

b. add function

```
(define add
  (lambda (p q)
    (cons (+ (car p) (car q))
          (+ (cdr p) (cdr q)))))
```